**Course Name:- PCCCS495: Advanced Programming(OOP)**

**Project Title:- Online Quiz Engine using Factory Pattern for Question Types**

**Student Name:- Writoja Boral**

**Roll Number:- 84**

**Instructors Name:- Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas**

**Semester:- 4th (Spring 2026)**

## 1.Problem Statement:

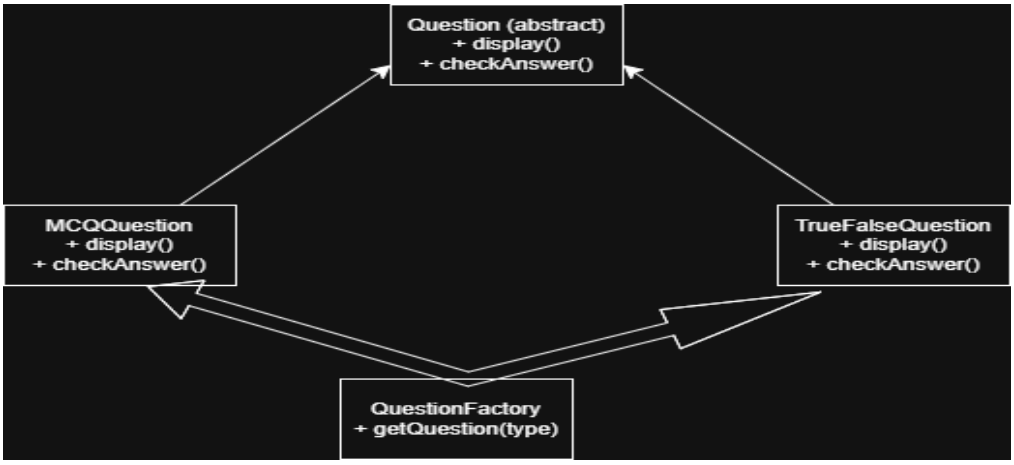
**In modern education systems, quizzes are widely used for evaluating knowledge. However, many basic quiz applications are not flexible and become difficult to extend when new types of questions are introduced. These systems often use rigid coding structures, making them hard to maintain.**

**The objective of this project is to develop an Online Quiz Engine that supports multiple types of questions such as Multiple Choice Questions (MCQ) and True/False questions. The system uses the Factory Design Pattern to dynamically create question objects based on user requirements.**

**This approach ensures scalability and flexibility. Additionally, input validation and error handling are implemented to improve reliability and user experience.**

## 2. System Design:

### 2.1 UML Class Diagram-



**2.2 Description of Major Classes-**

**Question (Abstract Class):**

**Defines the basic structure of all question types. It includes abstract methods like display() and checkAnswer().**

**MCQQuestion:**

**Represents multiple-choice questions. It displays options and checks selected answers.**

**TrueFalseQuestion:**

**Handles true/false type questions and validates responses accordingly.**

**QuestionFactory:**

**Responsible for creating objects of different question types based on user input. It hides object creation logic from the main program.**

**2.3 Package Structure-**

- **src/ – Contains all Java source files**
- **docs/ – Contains UML diagram**
- **report/ – Contains project report**
- **slides/ – Contains presentation slides**

**2.4 Interaction Flow-**

1. **Program starts execution**
2. **QuestionFactory creates question objects**
3. **Questions are stored in a list**
4. **Each question is displayed**
5. **User enters answers**
6. **Input is validated**
7. **Answers are checked**
8. **Score is updated**
9. **Final score is displayed**

## 3. OOP Concepts Demonstrated:

**3.1 Abstraction-**

**Where Used: Abstract class Question**
**Why Used: To define a common interface for all question types**

**Code Snippet:**

```
abstract class Question {
```

```
    abstract void display();
    abstract boolean checkAnswer(String answer);
}
```

**Explanation:**

Abstraction hides implementation details and ensures that all question types follow a common structure.

### 3.2 Inheritance-

Where Used: MCQQuestion and TrueFalseQuestion extend Question
Why Used: To reuse common properties and behavior

**Explanation:**

Inheritance allows subclasses to use features of the parent class, reducing code duplication.

### 3.3 Polymorphism-

Where Used: checkAnswer() method
Why Used: To allow different behavior for different question types

**Explanation:**

Each subclass implements its own version of checkAnswer(), enabling dynamic behavior.

### 3.4 Encapsulation-

Where Used: Data and methods inside classes

Why Used: To protect data and maintain modular design

## 4. Implementation Highlights:

### 4.1 Factory Pattern Implementation-

```
public class QuestionFactory {
    public class QuestionFactory {
      if (type.equalsIgnoreCase("MCQ"))
        return new MCQQuestion();
```

```
        else if (type.equalsIgnoreCase("TF"))
            return new TrueFalseQuestion();
        else
            throw new IllegalArgumentException("Invalid question type");
    }
}
```

**Justification:**
**The Factory Pattern separates object creation from the main logic, making the system flexible and easy to extend.**

**4.2 Input Validation-**

```
while (true) {
    ans = sc.nextLine();
    if (ans.isEmpty()) {
        System.out.println("Answer cannot be empty!");
        continue;
    }
    break;
}
```

**Justification:**
**Ensures that the user cannot submit empty input, improving robustness.**

**4.3 Scoring Logic-**

```
if (q.checkAnswer(ans)) {
    score++;
}
```

**Justification:**
**Tracks the number of correct answers and calculates final score.**

## 5. Testing & Error Handling:

### 5.1 Test Cases-

| Test Case | Input | Expected Output |
|---|---|---|
| Correct Answer | Valid input | Score increases |
| Wrong Answer | Incorrect input | Score unchanged |
| Empty Input | Blank | Error message |

### 5.2 Edge Cases

- Empty input
- Invalid question type
- Incorrect answer format

### 5.3 Failure Handling

- Prevents program crashes
- Uses exception handling
- Displays meaningful error messages

## 6. Git Workflow:

**The project was developed using a structured Git workflow with multiple meaningful commits.**

### Development Progression

- Initial project setup
- Added base classes
- Implemented Factory Pattern
- Added input validation
- Improved quiz logic
- Added UML diagram
- Completed README

```
MINGW64:/c/Users/bwrit/term-ii-project-submission-bwritoja07-lgtm

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$ git add .

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$ git commit -m "Added UML class diagram"
[main 2c74d0b] Added UML class diagram
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 docs/uml.png

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 22 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 25.43 KiB | 8.48 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/IEM-Gurukul/term-ii-project-submission-bwritoja07-lgtm.git
   79d39b3..2c74d0b  main -> main

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$
```

```
MINGW64:/c/Users/bwrit/term-ii-project-submission-bwritoja07-lgtm
   79d39b3..2c74d0b  main -> main

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$ git add .

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$ git commit -m "Completed README documentation"
[main 5998ea5] Completed README documentation
 1 file changed, 29 insertions(+), 43 deletions(-)

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 22 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 910 bytes | 910.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/IEM-Gurukul/term-ii-project-submission-bwritoja07-lgtm.git
   2c74d0b..5998ea5  main -> main

bwrit@Writaja_PC MINGW64 ~/term-ii-project-submission-bwritoja07-lgtm (main)
$
```

## 7. Conclusion & Future Scope:

**Conclusion-**

**The project successfully demonstrates the implementation of Object-Oriented Programming concepts along with the Factory Design Pattern. It improves flexibility, scalability, and maintainability of the system.**

**Future Scope-**

- **Develop a graphical user interface (GUI)**
- **Integrate database for storing questions**
- **Add more question types**
- **Implement timer-based quizzes**

**Appendix:**

**Dear Writoja Boral ,**

**Project: Online Quiz Engine using Factory Pattern for Question Types**
**Decision: APPROVED_MINOR**
**Score: 90**

**Strengths:**
**The proposal clearly identifies a relevant problem in online learning platforms and proposes a well-suited object-oriented solution. The explicit use of the Factory Design Pattern for managing different question types is an excellent demonstration of applying design patterns to enhance flexibility and scalability. The listed OOP concepts (Abstraction, Inheritance, Polymorphism) are directly applicable and central to the solution. Features are well-defined, practical, and align perfectly with the problem statement and target audience. The architectural description outlines a modular and coherent design.**

**Improvements:**
**1. **UML Diagrams:** Consider including (or stating intent to include) basic UML class diagrams to visually represent the question hierarchy and the factory pattern, which would greatly enhance clarity.**
**2. **User Interface (UI) Approach:** Briefly mention the intended approach for the user interface (e.g., console-based, simple Swing/JavaFX GUI) to provide a more complete picture of the application.**
**3. **Persistence Strategy:** How will quizzes and questions be stored and retrieved long-term? Adding a brief note on data persistence (e.g., serialization, text files, simple database) would make the solution more robust.**
**4. **Error Handling Details:** While 'Exception Handling' is listed, mentioning specific types of errors or scenarios where it will be crucial (e.g., invalid user input, file I/O issues) could add more depth.**
**5. **Testing:** A brief mention of a testing strategy (e.g., unit tests for question logic,**

**integration tests for quiz flow) would be beneficial.**

**Regards,**
**Course Instructor**

**PCCCS495 Proposal Review – APPROVED_MINOR**

Dear Writoja Boral ,

Project: Online Quiz Engine using Factory Pattern for Question Types
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly identifies a relevant problem in online learning platforms and proposes a well-suited object-oriented solution. The explicit use of the Factory Design Pattern for managing different question types is an excellent demonstration of applying design patterns to enhance flexibility and scalability. The listed OOP concepts (Abstraction, Inheritance, Polymorphism) are directly applicable and central to the solution. Features are well-defined, practical, and align perfectly with the problem statement and target audience. The architectural description outlines a modular and coherent design.

Improvements:
1. **UML Diagrams:** Consider including (or stating intent to include) basic UML class diagrams to visually represent the question hierarchy and the factory pattern, which would greatly enhance clarity.
2. **User Interface (UI) Approach:** Briefly mention the intended approach for the user interface (e.g., console-based, simple Swing/JavaFX GUI) to provide a more complete picture of the application.
3. **Persistence Strategy:** How will quizzes and questions be stored and retrieved long-term? Adding a brief note on data persistence (e.g., serialization, text files, simple database) would make the solution more robust.
4. **Error Handling Details:** While 'Exception Handling' is listed, mentioning specific types of errors or scenarios where it will be crucial (e.g., invalid user input, file I/O issues) could add more depth.

Shot on moto g64 5G
Shot by writoja